

(logo 图片)

网络安全培训教材

(出版)

(编写)

教材总目录

小学（初中、高中） 学段

(封面页)

【引言】

(教学内容，教学目的，教学意义简要总结)

第三单元：编程思维与基础

1.3 编程思维与基础

【引言】

本单元将带你走进编程的世界，但我们的起点并非复杂的代码。我们将首先解锁程序运行的底层逻辑——编程思维。随后，我们将亲自动手，从有趣的图形化积木编程到简洁的 Python 代码，指挥屏幕中的角色与画笔，将你的创意变为生动的数字作品。让我们一起，感受用逻辑构建世界的魅力。

【学习目标】

- 1 掌握顺序、循环、条件与事件这四种基本的编程思维，并能运用它们分解、描述生活中的简单问题。
 - 2 运用“问题分解”的方法，将复杂任务拆解为一系列清晰、可执行的步骤。
 - 3 在 Scratch 中，综合运用事件、控制、外观等模块，创作一个包含交互与分支情节的安全主题动画。
 - 4 在 Python 海龟绘图中，使用基础语法与绘图指令，创建可视化图形，直观感受从代码到作品的成就感。
-

【课程意义】

- 1、系统性地训练计算思维，学会像程序员一样思考，锻炼结构化分析问题、逻辑化解决问题的能力；
 - 2、体验从无到有、从逻辑到成品的完整创造过程，激发创新潜能与实践热情。
-

1.思维启蒙：顺序、循环、条件、事件

在本课中，我们将认识编程中的四种基本思维模式：

顺序：一步一步做已经设定好的内容；

循环：重复执行某段任务；

条件：根据不同情况做出选择.

事件：编程里每一句代码都对应一个事件

它们就像是编程的“积木”，所有的程序都是由这些“积木”组合而成的。

1.1. 顺序结构：一步一步来

什么是顺序？

顺序是指按照一定的步骤依次执行任务的思维方式。就像你每天早上的起床流程：起床 → 刷牙 → 洗脸 → 吃早餐 → 出门上学，这些步骤必须按顺序进行，不能颠倒。

程序中的语句通常也是按顺序执行的。

例如，你要让计算机完成“打招呼并询问姓名”的任务，代码可能是这样的：

```
//text
```

```
显示： “你好！ ”
```

```
显示： “你叫什么名字？ ”
```

```
等待用户输入名字
```

```
显示： “很高兴认识你， [名字]！ ”
```

如果打乱顺序，结果就会混乱，结果可能就会出差错。

再来一个简单的代码例子：

```
5→a;           //将数值 5 交给 a  
6→b;           //将数值 6 交给 b  
a + b;          //两个相加  
结果为：11
```

核心思想：做事之前先理清步骤，一步一步来，才能有条理的解决问题。

1.2. 循环：重复的事，交给循环

什么是循环？

循环思维是指让计算机重复执行某一段代码，直到满足某个条件才停止。

就像你每天都要吃饭、每周都要上学，这些是“重复性事件”。

在编程中，如果你想让计算机打印 10 遍“我要认真听讲！”，你不必写 10 行相同的代码，而是用一个循环：

```
//text  
  
重复执行 10 次：  
  
打印：“我要认真听讲！”
```

生活中也有很多循环的例子：音乐播放列表的循环播放、洗衣机设定的洗涤次数、你做俯卧撑时的计数等。

一个简单的代码例子：

```
x=10;           //数值 10 交给 x

while(x>0):     //当 x>0 的时候

    print ( "1" );    //向屏幕输出 1
```

在这个代码中，实现了一个一直输出 1 的功能。

里面用到的 while 则是一个循环常用到的关键字，print 则像计算机的嘴巴，可以把东西输出到屏幕上。

思维启发：学会识别重复性任务，用“循环”思维提高效率，避免无效劳动。

1.3. 条件：如果...那么...

什么是条件？

条件是指根据不同的情况做出不同的反应。就像你每天决定是否带伞：

```
如果 天气预报说下雨：

    带伞；

否则：

    不带伞；
```

在编程中：

条件语句可以让代码自动进行判断，如下一个简单的例子。

```
x=2;           //把 2 放到 x 中

if (x>0) :     //如果 x 中的数字大于 0

    x=1;       //把 "1" 放到 x 中

else:          //否则
```



```
x=0    //把 "0" 放到 x 中

//结果 x 的值为 0
```

在这个例子中，先让 x 的值为 2，条件语句则是：当 $x > 0$ 时，让 x 的值为 1，当 $x \leq 0$ 时让 x 的值为 0。

1.4. 事件：当...就...

什么是事件：

简单来说就是“当某事发生，就做出相应的反应”。

如果你早上听到了闹铃声，然后你起床，这个听到闹铃声然后起床的过程，被称为一个事件。

在编程中，每一个事件组合起来并发生就可以让程序“动起来”。从上面的例子看，每一个单独的语句都可以当成一个事件，由于程序中的指令要按顺序执行，所以每一个语句要执行的都是“当上一条语句执行完毕时开始执行”。

当我们运行一个软件时，我们的每个动作也可以当成是一个事件，比如在使用软件，我们点击 IDE 运行，或者点击 QQ 登录，这些也可以当做是一个事件。

1.5. 【实践】

1. 尝试用伪代码的形式把你在学校当中的事情按时间线排列
2. 选择一个重复的场景，并用循环语句描述他

2. 问题分解：如何吃掉一头大象？

面对一个复杂程序就像面对一头大象，如何才能“吃掉”它？本小节将带你掌握程序员的核心思维——问题分解。你将学会如何将庞大任务化整为零，把模糊想法变成清晰的行动蓝图。

2.1. 程序员的最大秘密：一切从分解开始

同学们，当你看到一款好玩的游戏或一个有用的 APP 时，你是否想过，程序员是如何开始创造它们的？

他们不会一开始就写下成千上万行复杂的代码。相反，他们做的第一件事，和我们面对一个巨大任务时一样：进行问题分解。

在编程中，问题分解有一个更专业的名字：“模块化设计”或“算法设计”。其核心思想就是那句古老的谚语：

“如何吃掉一头大象？一次吃一口。”

在编程中，这意味着：任何一个复杂的软件功能，都可以被拆解成一系列简单的、可以由计算机逐步执行的指令集合。

2.2. 编程中的“问题分解”为什么如此重要？

1. 计算机只懂步骤：计算机非常“笨”，它只能严格执行一条条清晰的指令。我们必须把模糊的目标（如“做个游戏”）变成它懂的“步骤语言”。

2. 便于编写和测试：一次只解决一个小问题，就像一次只拼一块乐高，写代码和找错误都更容易。

3. 实现团队协作：在大型项目中，不同的程序员负责不同的“小

模块”，最后像搭积木一样组合成完整的程序。

2.3. 实战：从派对策划到程序编写

让我们通过一个例子，看看同一个任务，在生活和编程中是如何被分解的。

【终极任务】：制作一个“自动生日祝福程序”

第 1 步：定义最终目标（明确程序功能）

生活类比：策划一场成功的生日会。

编程目标：创建一个程序，运行后能完成询问姓名、播放生日歌、显示祝福语这一系列动作。

第 2 步：寻找核心模块（识别主要功能块）

生活类比：将派对分解为“食物”、“装饰”、“活动”等模块。

编程分解：我们将“生日祝福程序”分解为三个主模块：

模块 A：输入与问候（获取用户信息）

模块 B：播放生日音乐（营造气氛）

模块 C：显示祝福画面（输出核心内容）

第 3 步：继续分解至具体指令（编写算法步骤）

现在，我们对每个编程模块进行更细致的分解，直到每一步都清晰到可以写成代码。

以“模块 A：输入与问候”为例：

指令 1：在屏幕上显示文字：“请问你叫什么名字？”

指令 2：等待用户从键盘输入他们的名字，并记住它。

指令 3：将用户的名字和“你好！”组合成一句话，并显示出来。

(例如: “小明, 你好!”)

这个过程, 就是在设计 “算法” ——解决问题的准确步骤集合。

第 4 步: 排序与组合 (构建程序结构)

生活类比: 先准备食物, 再布置场地, 最后进行活动。

编程执行: 我们必须确定模块的执行顺序, 并把它们组合起来。

合理的顺序: A (输入问候) → B (播放音乐) → C (显示祝福)

这就是程序的 “逻辑流程” 。

2.4. 思维进阶: 你的第一个 “程序” 设计图

编程挑战: 设计一个 “早晨起床提醒程序”

假设你要为机器人设计一个程序, 让它能在早上帮你做好提醒。
请尝试分解这个任务。

参考示例:

最终目标: 机器人成功完成早晨提醒任务。

核心模块:

- 1. 时间监控模块**
- 2. 语音提醒模块**
- 3. 天气播报模块**

细化指令 (请为 “语音提醒模块” 设计步骤) :

- 指令: 检测到设定时间到。
- 指令: 用扬声器播放一段 “起床铃声音频” 。
- 指令: 用语音合成功能说: “早上好! 现在是 7 点整, 该起床了!”

4. 指令：等待 5 分钟，如果检测到人还未起床，重复步骤 2 和 3。

2.5. 核心概念总结：编程就是“翻译”

分解 步骤	生活中的问题分解	编程中的问题分解	核心概念
1	明确要“吃掉哪头大象”	定义 程序功能	需求分析
2	把大象分成几大块	拆分成 功能模块	模块化设计
3	把肉切成一口大小	细化为一步步 指令	算法设计
4	决定先吃哪一口	确定代码 执行顺序	程序结构

记住：编程，就是将人类想要解决的问题，通过“问题分解”这道工序，翻译成计算机能听懂的“步骤语言”的过程。

课后实践：

找一件你明天必须做的事情（例如：“完成数学作业”或“整理书包”）

1. 用本节学的四步法对它进行分解。
2. 想象你正在教一个机器人完成这件事，把你的分解步骤写成一份“指令清单”。
3. 思考一下，你的清单里，有哪些步骤对机器人来说还是太复

杂了，需要继续分解？（例如：“完成数学作业”需要分解为“读题”、“计算”、“写答案”、“检查”）

3.编程实践（Scratch）：

--创作安全主题互动动画

前面课程简单学习了网络威胁与计算机编程思想，本节课将开启一次全新的实践——用 Scratch 编程工具，创作一个安全主题互动动画。

Scratch 是由美国麻省理工学院媒体实验室终身幼儿园团队开发的图形化编程工具。它将复杂的程序代码封装成色彩鲜艳、功能明确的**积木指令块**，学习者只需像拼接乐高积木一样拖动组合，即可创造出交互式故事、动画和游戏。



这种设计使得编程学习不再枯燥，而是充满了创造与探索的乐趣，

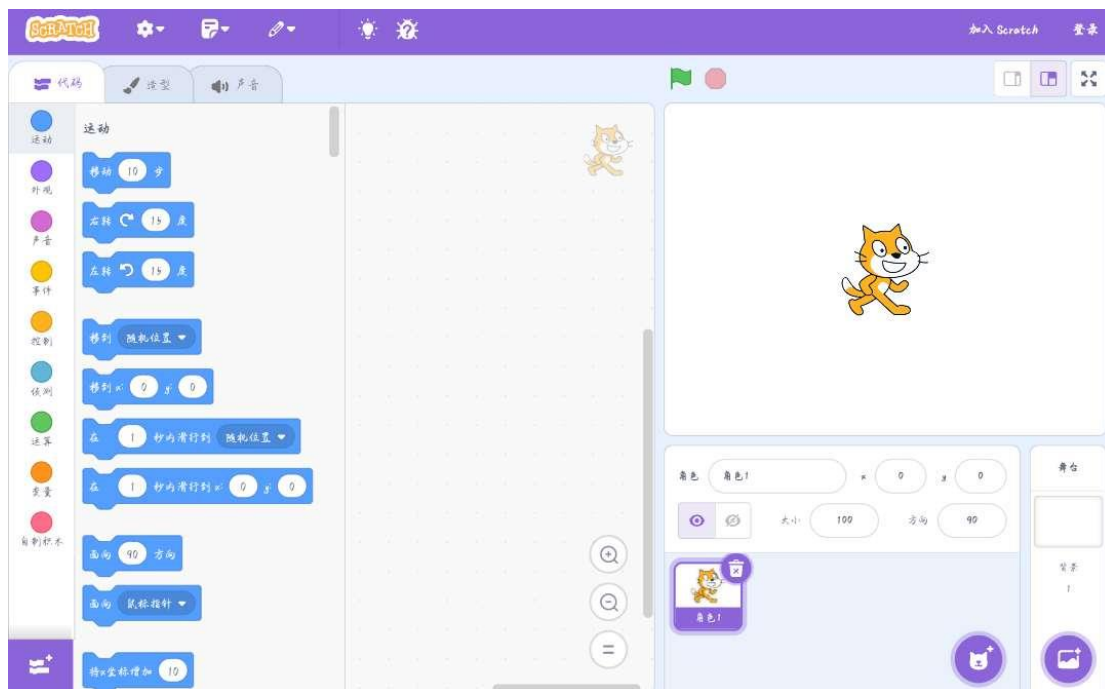
是培养编程思维与数字创造力的理想起点。

学习本课，意味着同学们将从知识的接收者，转变为内容的**主动创造者**。通过亲手搭建动画情节，网络安全知识将不再是书本上的条文，而是生动场景中的判断与行动。

3.1. 认识 Scratch 创作环境

启动 Scratch 在线编辑器 (<https://scratch.mit.edu/>) 后，呈现的主界面主要分为四个区域：

- [1] 舞台区：位于界面右侧，是程序运行结果的展示窗口。绿旗用于启动程序，红色按钮用于停止。
- [2] 角色列表区：位于舞台区下方，管理项目中的所有角色，可进行添加、删除、重命名等操作。
- [3] 积木指令区：位于界面中部，按功能（如运动、外观、声音、事件、控制等）分类存放所有可用的编程积木。
- [4] 脚本编辑区：位于界面左侧，是拼接积木、构建程序逻辑的主要工作区域。



3.2. Scratch 互动动画通用创作流程

第一阶段：规划与设计

[1]明确核心交互

定义互动点：首先确定动画中用户可以在哪里进行干预。是点击某个角色？还是通过键盘按键？或是在询问框中输入选择？

规划分支逻辑：明确用户的不同互动行为将触发何种不同的结果。这构成了动画的叙事分支。

[2]拆解动画序列

将整个动画故事分解为一个个连续的“情节”或“状态”。例如：“起始状态” -> “事件触发” -> “角色 A 反应” -> “用户选择” -> “结局 A/结局 B”。

为每个“情节”明确：哪些角色需要出场？他们需要执行什么动作（移动、说话、改变造型）？这个情节如何切换到下一个？

[3]设计角色与消息体系

根据情节需要,列出所有必要的角色,并规划他们的初始状态(显示/隐藏、位置、造型)。

设计“广播”消息列表。将每个情节的转换点用一条特定的广播消息来标识(例如:“游戏开始”、“挑战出现”、“成功”、“失败”)。这是协调多个角色行动的核心。

第二阶段: 编程与搭建

[1]搭建程序骨架(初始化)

为每个角色编写以 当绿旗被点击 开头的初始化脚本。

在此脚本中,设定好角色的初始位置、造型、大小以及显示/隐藏状态,确保每次动画都能从一个确定的起点开始。

[2]实现事件驱动逻辑

使用 当角色被点击、当按下空格键 或 广播 作为触发器,来启动核心互动。

关键技巧:对于需要用户点击的角色,其脚本通常以 当角色被点击 开始,然后内部使用 广播 来通知其他角色剧情已推进。

[3]构建叙事流与分支

大量使用 当接收到 [消息] 积木。每个角色都应监听与自己相关的消息,并在收到后执行相应的动作序列(移动、说话、切换造型等)。

在需要用户决策的地方,使用 询问 和 如果...那么...否则 积木。根据回答,广播两条不同的消息(如“选择 A”、“选择 B”),从而触发两条独立的叙事分支。

[4]控制节奏与流程

在连续的动作之间插入 等待...秒 积木，确保动画有舒适的观看节奏，避免所有事件瞬间发生。

使用 重复执行 积木来创建循环动画，如角色行走、物体闪烁等。

第三阶段：测试与迭代

[1]模块化测试

不要等全部编完再测试。每编写完一个功能模块（如一个角色的出场动画、一个交互逻辑），就立即运行测试，确保其行为符合预期。

[2]集成测试与调试

流程测试：从头到尾运行整个动画，检查各情节是否按设计顺序无缝衔接。

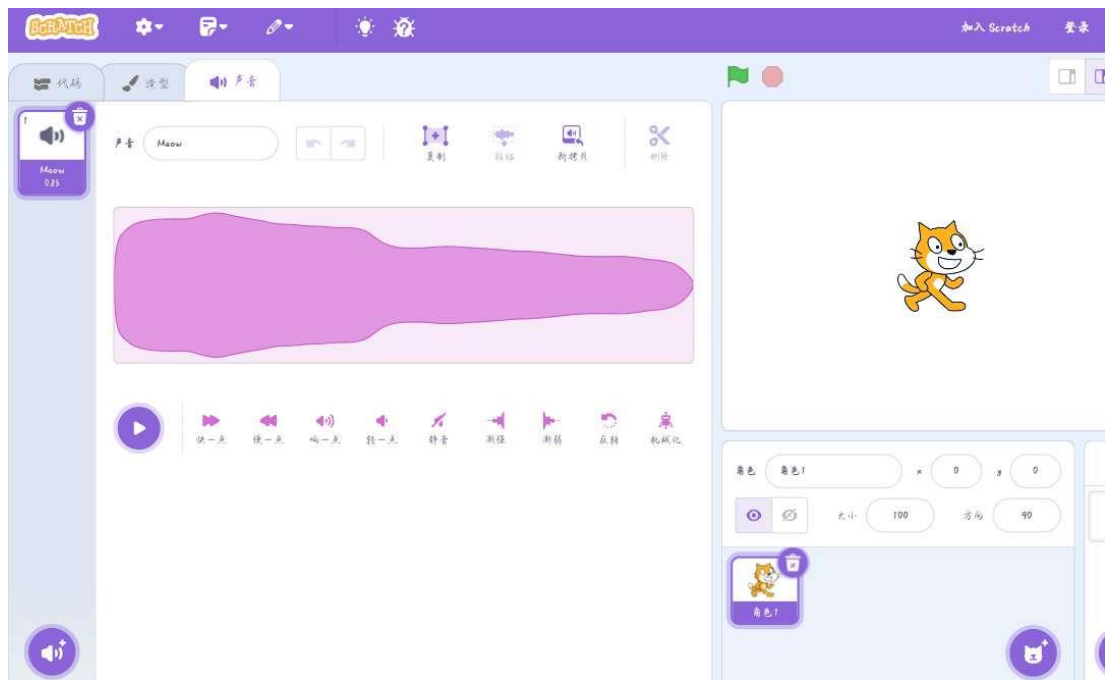
分支测试：遍历所有可能的用户选择路径，确保每个分支都能正确触发并结束，没有逻辑死角。

消息链路验证：确认每一个“广播”都能被正确的角色接收并处理。

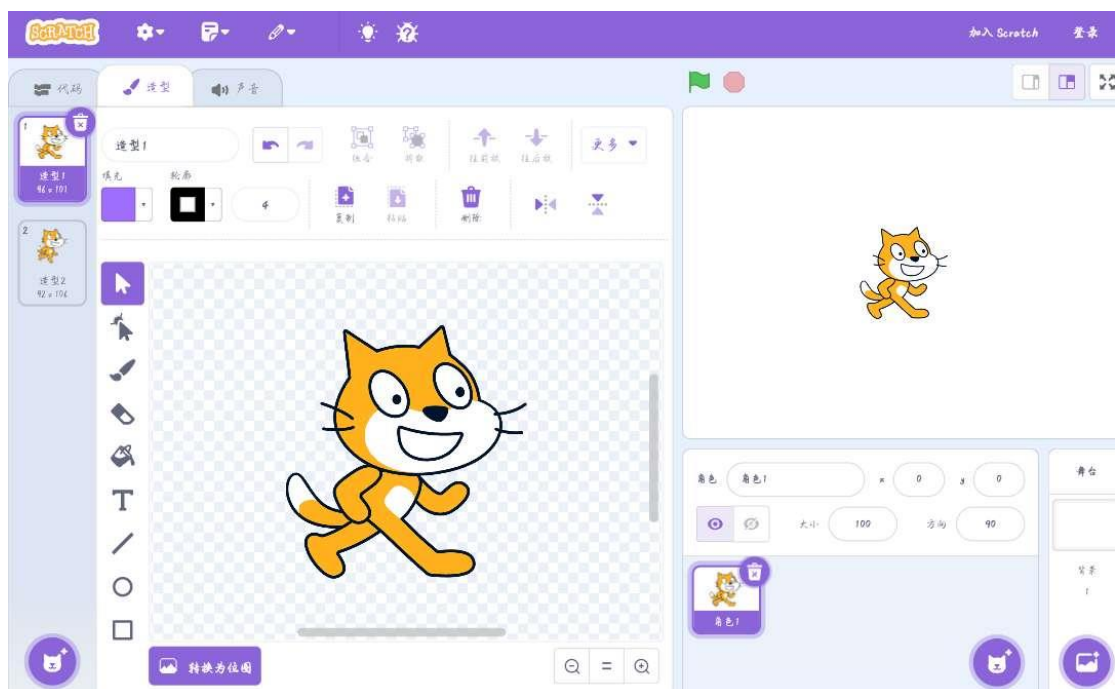
第四阶段：优化与打磨

[1]增强表现力

音效：为关键动作（如点击、成功、失败）添加音效，能极大提升互动感和反馈感。



视觉丰富：为角色设计多个“造型”，通过下一个造型 或 切换为[某]造型 来表现状态变化。使用 外观 中的特效（如颜色、虚像、像素化）来突出重要时刻。



界面与反馈：可以考虑添加“开始界面”、“结束界面”或“得分”等元素，让作品更完整。

[2]最终审查与分享

进行最后一次完整审查，优化细节。

为作品起一个恰当的名字，并编写项目说明，然后分享给他人体验和反馈。

4.编程实践 (Python 海龟绘图):

--感受代码的魅力

本小节聚焦 Python 海龟绘图这一趣味编程工具，旨在让你从“看懂代码”进阶到“创造图形”——通过直观的代码指令控制虚拟“画笔”移动、上色，在实践中巩固变量、循环、条件判断等基础语法，同时感受代码与视觉创意结合的魅力。

4.1. Python 初体验：与计算机对话的语言

(1) Python 是什么？

Python 是一种高级编程语言，以简洁、易读的语法而闻名。它就像是英语和数学的结合体，让我们能用近乎自然语言的方式告诉计算机该做什么。

Python 的特点：

简单易学：语法清晰，接近英语

功能强大：从网页开发到人工智能都能胜任

免费开源：任何人都可以免费使用和分享

(2) 搭建 Python 环境

要运行 Python 代码，你需要一个“工作台”：

在线环境：Replit、PythonTutor（无需安装）

本地环境：安装 Python + IDLE（官方自带）或 PyCharm、VS Code 等专业工具

Tip：对于初学者，推荐使用在线环境，快速开始编程体验~

4.2. Python 基础语法：编程的"ABC"

学习 Python 就像学习一门新语言，需要从最基本的"单词"和"句型"开始。下面我们将通过一系列贴近生活的例子，来掌握这些构建所有程序的基础积木。

(1) 变量：数据的"储物柜"

概念：变量就像是一个贴了标签的储物柜，你可以在里面存放数据（比如数字、文字），之后通过标签名就能随时找到和使用这个数据。

生活中的例子：想象你有一个名叫"我最爱的水果"的储物柜。今天你可以在里面放"苹果"，明天你可以取出"苹果"放进"香蕉"。柜子的名字没变，但里面的东西可以改变。

```
1 # 创建变量并赋值
2 name = "小明"      # 字符串类型，存储文本
3 age = 12           # 整数类型，存储数字
4 height = 1.55      # 浮点数类型，存储小数
5 is_student = True  # 布尔类型，存储真/假
```

变量是编程中最基础也最重要的概念。你可以把变量想象成一个带标签的储物柜——标签就是变量名（如 name），柜子里存放的东西就是变量的值（如"小明"）。在 Python 中创建变量非常简单，只需要使用等号=进行赋值即可，Python 会自动根据你赋予的值来确定数据的类型。

变量名应该具有描述性，让别人一看就知道这个变量用来存储什么。比如用 student_name 而不是 a 或 x。另外，Python 中的变量是动态类型的，这意味着同一个变量可以在程序的不同时刻存储不同类型的数据，这为编程提供了很大的灵活性。

解读:

- = 不是数学中的"等于", 而是"赋值"的意思, 好比把右边的物品放进左边的柜子。
- 变量名 (如 my_name, age) 最好取得一目了然, 就像给储物柜贴上一个清晰的标签。
- 变量里的内容是可以改变的, 最后一次存放的东西才是它当前的值。

(2) 输入与输出: 与程序的"对话"

概念: print() 是程序的"嘴巴", 负责说话 (输出信息); input() 是程序的"耳朵", 负责听你说话 (接收输入)。

生活中的例子: 这就像你和智能音箱的对话。你说"小爱同学, 今天天气怎么样?" (这是 input()), 它回答"今天晴, 气温 25 度" (这是 print())。

```
1 # 输出信息到屏幕
2 print("你好, 世界!")
3 print("我的名字是", name, ", 今年", age, "岁")
4
5 # 从用户获取输入
6 user_name = input("请输入你的名字: ")
7 user_age = input("请输入你的年龄: ")
8 print("欢迎你, ", user_name, "! ")
```

输入输出是程序与用户交互的桥梁。print()函数就像程序的"嘴巴", 负责把信息"说"给用户听; 而 input()函数则像程序的"耳朵", 负责"听"用户说的话。这两个函数的组合让程序不再是冷冰冰的代码, 而是能够与人进行对话的智能工具。

需要注意的是, input()函数获取的所有输入都会被当作字符串处理。如果你需要数字, 需要使用 int()或 float()进行类型转换, 比如

age = int(input("请输入年龄: "))。print()函数可以同时输出多个内容，用逗号分隔，Python 会自动在它们之间添加空格。

解读：

- print() 可以输出文字（需要用引号括起来），也可以输出变量里的值。
- input() 会让程序暂停，等待你输入。它还可以在括号里显示一句提示语。
- **关键点：**从 input() 得到的数据是"字符串"（一串字符），如果想进行数学计算，必须用 int()（转换为整数）或 float()（转换为小数）进行转换。

(3) 条件判断：程序的"选择题"

概念：让程序能够根据不同情况，做出不同的反应。核心是 "如果...那么...否则..." 的思维。

生活中的例子：如果你考试分数大于等于 90 分，那么妈妈会奖励你；否则，你需要继续努力。

```
10     # if-else 语句
11     score = 85
12
13     if score >= 90:
14         print("优秀!")
15     elif score >= 70:
16         print("良好")
17     else:
18         print("需要加油")
```

条件判断让程序具备了基本的"思考"能力。就像我们在生活中根据不同的情况做出不同的选择一样，程序通过 if-elif-else 结构来根据条件执行不同的代码块。这种结构让程序能够处理复杂的情况，而

不仅仅是机械地执行固定步骤。

条件判断的核心是条件表达式，如 `score >= 90`，它的结果只能是 True（真）或 False（假）。Python 会按顺序检查每个条件，一旦找到第一个为 True 的条件，就执行对应的代码块，然后跳过其他所有条件。特别注意，在 Python 中代码块的划分是通过缩进（通常是 4 个空格）来实现的，这是 Python 语法的一个重要特点。

解读：

- `if`、`elif`、`else` 是进行条件判断的关键词。
- **缩进是 Python 的语法规则：**隶属于每个条件分支的代码，必须**向右缩进（通常是 4 个空格）**。这就像写大纲一样，表示这些代码是受上面那个条件控制的。
- `==` 是"比较两者是否相等"，而 `=` 是"赋值"，千万不能混淆。
- 程序会从上到下判断条件，一旦某个条件满足，就会执行对应的代码块，然后跳过剩下的所有 `elif` 和 `else`。

(4) 循环结构：程序的"重复劳动"

概念：让一段代码可以被重复执行多次，从而高效地处理重复性任务。

生活中的例子：

- **for 循环：**就像老师让你"把'我要认真听讲'这句话抄写 10 遍"，你清楚地知道要重复多少次。
- **while 循环：**就像妈妈让你"只要天还没黑，就继续在外面玩"，你不知道具体能玩多久，但你知道循环停止的条件是"天黑"。

```
20 # for循环: 重复固定次数
21 for i in range(5): # 重复5次
22     print("这是第", i+1, "次循环")
23
24 # while循环: 满足条件时重复
25 count = 1
26 while count <= 3:
27     print("计数: ", count)
28     count = count + 1 # 重要: 改变条件, 避免无限循环
```

循环结构是编程中提高效率的利器。想象一下, 如果需要对计算机从 1 数到 1000, 没有循环的话就要写 1000 行 print 语句! for 循环适合在知道具体重复次数的情况下使用, 而 while 循环则在需要满足某个条件时才停止重复的情况下更加适用。

for i in range(5)中的 i 是一个循环变量, 它在每次循环中会取不同的值 (从 0 到 4)。range(5)生成一个从 0 到 4 的序列。while 循环要特别注意设置循环终止条件, 比如 count = count + 1 确保循环不会无限进行下去。在编程中, 我们称之为"避免死循环"。

解读:

- **for 循环:** 通常用于**已知循环次数**的场景。i 是一个计数器变量, 每次循环会自动变化。
- **while 循环:** 用于**未知循环次数**, 但**已知循环条件**的场景。必须确保循环条件最终能变为"不成立", 否则程序会永远循环下去 ("死循环")。
- **缩进同样重要:** 需要被重复执行的代码块, 必须正确缩进。

【Python编程核心概念总结】		
概念	作用	示例
变量	存储数据	<code>name = "小明"</code>
数据类型	定义数据种类	字符串、整数、浮点数、布尔值
输入输出	与用户交互	<code>input()</code> , <code>print()</code>
条件判断	根据不同情况执行不同代码	<code>if-elif-else</code>
循环	重复执行代码	<code>for</code> , <code>while</code>
函数	封装可重用代码	<code>t.forward(100)</code>
模块	扩展Python功能	<code>import turtle</code>

4.3. 海龟绘图入门：代码的"画笔"

(1) 引入绘图模块

```
1 # 导入海龟绘图模块
2 import turtle
3
4 # 创建画布和海龟
5 screen = turtle.Screen()
6 screen.title("我的第一个Python绘图")
7 t = turtle.Turtle()
```

(2) 基础绘图命令

```
1 # 移动命令
2 t.forward(100) # 前进100像素
3 t.backward(50) # 后退50像素
4 t.left(90) # 左转90度
5 t.right(45) # 右转45度
6
7 # 画笔控制
8 t.penup() # 抬起画笔，移动不画线
9 t.pendown() # 落下画笔，移动画线
10 t.pencolor("red") # 设置画笔颜色
11 t.pensize(3) # 设置画笔粗细
```

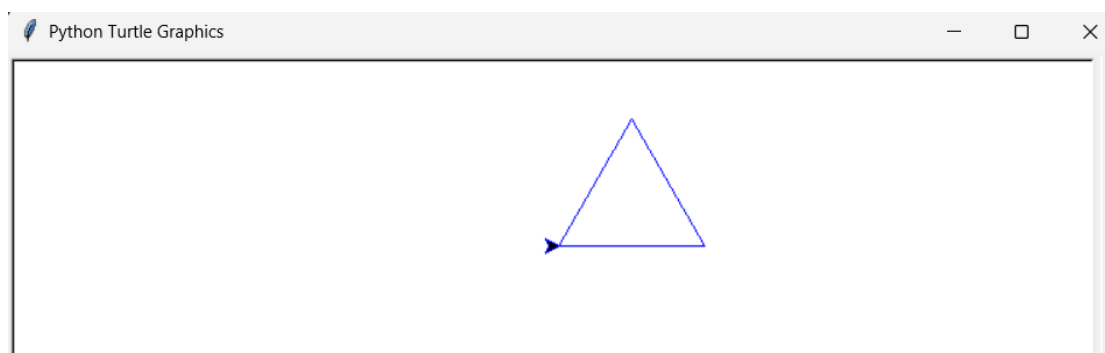
【实践练习 1】：绘制一个三角形

```

1  import turtle
2
3  t = turtle.Turtle()
4  t.pencolor("blue")
5
6  # 绘制三角形
7  for i in range(3):
8      t.forward(100)
9      t.left(120)  # 三角形内角和180度，每次转120度
10
11  turtle.done()

```

运行结果：一个蓝色的等边三角形



4.4. 综合实践：创意图形设计

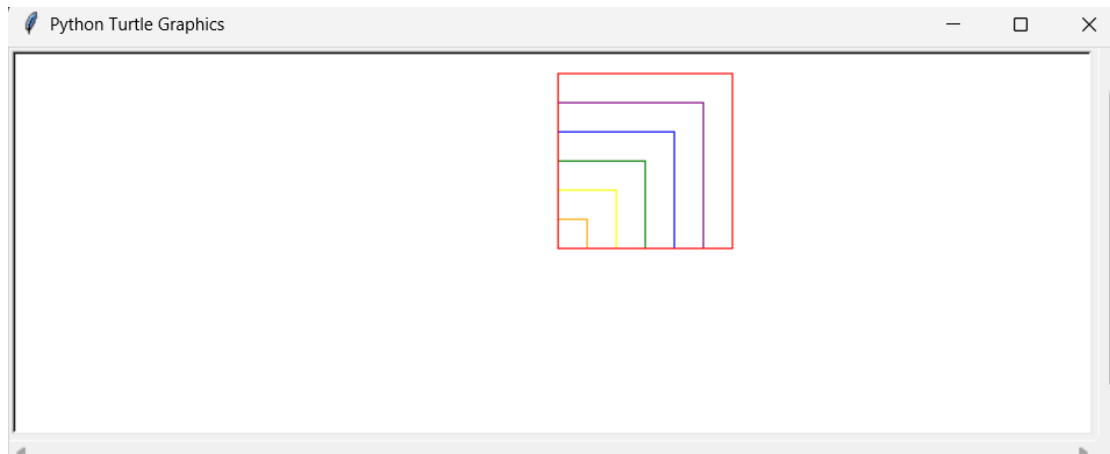
(1) 多彩正方形序列

```

1  import turtle
2
3  t = turtle.Turtle()
4  t.speed(0)  # 最快速度
5
6  colors = ["red", "orange", "yellow", "green", "blue", "purple"]
7
8  # 绘制逐渐变大的彩色正方形
9  for size in range(20, 121, 20): # 从20开始，每次增加20，直到120
10     t.pencolor(colors[(size//20) % 6]) # 根据大小选择颜色
11     for i in range(4):
12         t.forward(size)
13         t.left(90)
14
15  t.hideturtle()
16  turtle.done()

```

运行结果：一组嵌套的彩色正方形



(2) 交互式星星绘制器

```
1 import turtle
2
3 # 设置画布
4 screen = turtle.Screen()
5 screen.title("交互式星星绘制器")
6 screen.bgcolor("black") # 黑色背景
7
8 t = turtle.Turtle()
9 t.speed(0)
10 t.pencolor("yellow")
11
12 # 获取用户输入
13 points = int(input("请输入星星的角数 (5-10): "))
14 size = int(input("请输入星星的大小 (50-200): "))
15
16 # 绘制星星
17 angle = 180 - (180 / points) # 计算星星的转向角度
18
19 for i in range(points):
20     t.forward(size)
21     t.right(angle)
22
23 t.hideturtle()
24 turtle.done()
```

运行结果：一组嵌套的彩色正方形

```
请输入星星的角数 (5-10): 6
请输入星星的大小 (50-200): 100
```



